KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

PERSONAL CONTACT DIRECTORY WITH SEARCH AND UPDATE FEATURES

Submitted by

SAMVARDHINI K

25BIT142

Under the Guidance of

MUGILAN R

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

Error! Number cannot be represented in specified format.

DECLARATION

I hereby declare that the project entitled "**PERSONAL CONTACT DIRECTORY WITH SEARCH AND UPDATE FEATURES"** submitted in partial fulfillment of the requirements for the course **Java Programming** is a **record of my original work** carried out under the guidance of **MUGILAN R** ,Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

CERTIFICATE

This is to certify that the project entitled **"PERSONAL CONTACT DIRECTORY WITH SEARCH AND UPDATE FEATURES"** is a **bona fide work** carried out by **SAMVARDHINI K, B.Sc IT 'B'(2025-2028) INFORMATION TECHNOLOGY, KG College of Arts&Science**, in partial fulfillment of the requirements for the course **Java Programming**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore.

Date:

Submitted for the Viva-Voce Examination held on

_____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

Error! Number cannot be represented in specified format.

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

Error! Number cannot be represented in specified format.

ABSTRACT

The **Personal Contact Directory with Search and Update Features** is a Java-based application designed to help users store and manage their personal contact information in an organized manner. In many situations, people maintain contact details manually in notebooks or scattered digital notes, which can lead to difficulty in finding specific contacts, outdated information, and poor record management. This project aims to provide a simple and efficient system for storing, searching, and updating contact details in a structured format.

The system allows users to add contact information such as **name, phone number, email address, and other relevant details.** It also provides a **search feature** that helps users quickly find a contact by entering the name or phone number. Additionally, the **update feature** enables users to modify existing contact details whenever changes occur, ensuring that the information remains accurate and up to date.

The application is developed using **Core Java programming concepts**, including **classes**, **objects, arrays, methods, loops, and conditional statements.** The program can be implemented as a **console-based application** or enhanced with a graphical user interface using **Java Swing.**

The main objective of this system is to **improve contact management, provide quick access to stored information, and reduce the effort required to maintain personal directories manually.** This project demonstrates the practical use of programming concepts in Java and provides a basic foundation for building more advanced contact management systems in the future.

1. INTRODUCTION

In today's **digital world,** managing **personal contacts efficiently** has become very important. Many people still store **contact details** such as **phone numbers, addresses, and email IDs** manually in notebooks or simple phone lists. However, **manual contact management** can lead to several issues such as difficulty in **searching contacts, updating information,** or maintaining **organized records**. With the help of **computer technology, digital contact management systems** can simplify the process of storing, searching, and updating personal contacts. A **computerized contact directory** allows users to maintain a **structured list of contacts** and perform operations such as **adding, searching, updating, and deleting** contact information easily. The **Personal Contact Directory with Search and Update Features** is a **Java-based application** designed to store and manage **contact details efficiently**. The system allows users to **add new contacts, search for existing contacts** using names or phone numbers, and **update contact details** whenever required. This application provides a simple and organized way to maintain **contact information** such as **name, phone number, email address, and address.** The program helps users quickly access stored contact details without searching manually through large lists. The project is developed using **Core Java programming concepts** such as **classes, objects, arrays, methods, loops, and conditional statements.** The system can be implemented either as a console-based application or with a graphical user interface using Java Swing. The main objective of this system is to provide a simple and efficient way to manage personal contacts digitally while demonstrating the practical use of Java programming concepts in developing real-world applications.

1.1 Objective

The main objective of the **Personal Contact Directory with Search and Update Features** is to develop a simple Java-based application that allows users to manage personal contact details efficiently.

The specific objectives of this project are:

. To **design and develop** a Java-based contact directory system.

.To **store personal contact details** such as name, phone number, email, and address.

. To allow users to **search for contacts quickly**.

* To allow users to **update contact information easily.**
* To **reduce the difficulty of managing contacts manually.**
* To demonstrate the use of **Core Java concepts** such as arrays, loops, classes, and methods.

1.2 Overview

The **Personal Contact Directory with Search and Update Features** is a menu-driven application developed using the Java programming language. The system allows users to store and manage personal contact information efficiently.

The application allows users to perform operations such as:

* **Adding new contact details**
* **Searching for contacts by name or phone number**
* **Updating existing contact information**
* **Displaying all stored contacts**

The system maintains a structured list of contacts that can be easily accessed and modified. Users can quickly find a contact without manually checking multiple records.

This project demonstrates how Java programming can be used to develop simple information management systems that can be useful in daily life.

1.3 Features

The Personal Contact Directory System includes the following features:

* **Add Contact** – Allows the user to add **new contact details** such as name, phone number, email, and address.
* **Search Contact** – Enables users to search for contacts using the **name or phone number**.
* **Update Contact** – Allows modification of **existing contact details.**
* **Display Contacts** – Shows the list of all **saved contacts.**
* **User-Friendly Interface** – Provides a simple **menu-driven system** for easy use.
* **Fast Contact Retrieval** – Quickly finds contact information without manual searching.

2. PROGRAM ANALYSIS

Program analysis is an important phase in **software development** where the **problem is studied** carefully and the **requirements of the system are identified.** This phase helps in understanding how the system should function and what features are needed to solve the problem effectively. The analysis stage focuses on identifying problems in the existing system and developing a computerized solution that improves efficiency and organization.

2.1 Problem Definition

Many people still maintain personal contact information manually using notebooks or unorganized digital lists.

This manual system creates several problems such as:

* Difficulty in **searching for specific contacts**
* Difficulty in **updating contact information**
* Risk of **losing important contact details**
* Lack of **organized storage** for contact records
* **Time-consuming process** to find specific information

To overcome these problems, a **computerized contact directory system** is required.

The Personal Contact Directory with Search and Update Features provides a digital solution where users can easily store, search, and update contact information.

2.2 Project Overview

The Personal Contact Directory System is a **Java-based application** developed to help users manage personal contact information effectively.

The system provides a menu-driven interface where users can perform various operations on stored contacts.

The main functionalities include:

* Adding new contact details
* Searching contacts by name
* Updating existing contact information

* Displaying stored contacts

This project is implemented using Core Java programming concepts and can be developed as a console-based or graphical application.

2.3 Scopes and Limitations

Scope

The scope of the Personal Contact Directory System includes:

* **Storing personal contact details**
* **Searching contacts** quickly
* **Updating contact information**
* Displaying stored contacts
* Demonstrating **Java programming concepts**

The system can be used for:

* Personal contact management
* Student contact directories
* Small office contact records
* Simple address book applications

Limitations:

* The system **stores contacts only during program execution** (if no database is used).
* Limited number of contacts can be stored depending on the program structure.
* Advanced features such as cloud storage or synchronization are not included.

3. SYSTEM DESIGN

System design is the phase where the structure and components of the system are planned before coding begins. The Personal Contact Directory System is designed as a menu-driven application where users can perform operations such as adding, searching, updating, and displaying contacts. The system is divided into different modules to handle specific tasks.

3.1 Modules

1. Add Contact Module

This module allows users to add new contact information.

The details stored include:

*** Name**

*** Phone Number**

*** Email Address**

*** Address**

The entered information is stored in the contact directory.

2. Search Contact Module

This module allows users to search for a contact using the **name or phone number.**

Functions include:

* Accepting search input from the user
* Checking stored contacts
* **Displaying matching contact** details

3. Update Contact Module

This module allows users to **modify existing contact information.**

Users can update:

*** Phone number**

*** Email address**

*** Address**

This feature helps keep contact information up to date.

4. Display Contacts Module

This module displays all **stored contacts in an organized format**.

Example Output:

Name: Arun Kumar

Phone: 9876543210

Email: arun@gmail.com

Address: Chennai

5. Exit Module

This module **allows the user to exit the application safely.**

4. METHODOLOGY

Methodology explains the process followed to develop the project.

The development process includes the following stages:

*** Problem Analysis**

*** System Design**

*** Program Development**

*** Testing and Validation**

*** Result and Documentation**

4.1. Problem Analysis

In this phase, the need for a digital contact management system was studied.

The main problems identified were:

* Difficulty in **managing contacts manually**
* **Time-consuming search process**
* **Difficulty updating contact information**

Based on these problems, the system requirements were defined.

4.2. System Design

In this stage, the system structure was designed.

The application includes the following modules:

*** Add Contact**

*** Search Contact**

*** Update Contact**

*** Display Contacts**

*** Exit System**

4.3. Program Development

The program was developed using the **Java programming language.**

During development, the following steps were followed:

* Creating **classes and methods**

* Writing code to store contact details
* Implementing search and update features
* Designing a menu-driven interface

4.4. Testing and Validation

Testing ensures that the system works correctly.

Example Test Cases:

Test Case	**Expected Result**
Add Contact	Contact stored successfully
Search Contact	Correct contact displayed
Update Contact	Correct total displayed
Display Contacts	Contact updated correctly

All contacts shown properly.

4.5. Result and Documentation

After testing, the results were documented. The system successfully performs operations such as adding, searching, and updating contacts.Screenshots of program execution can be included to demonstrate the system functionality.

4.6. Algorithm

Algorithm for the **Personal Contact Directory System**:

1.Start the program
2.Display the menu options
3.Select an option (Add/Search/Update/Display)
4.If Add Contact → Enter contact details

5.If Search Contact → Enter name or phone number

6.If Update Contact → Modify existing details

7.If Display Contacts → Show all stored contacts

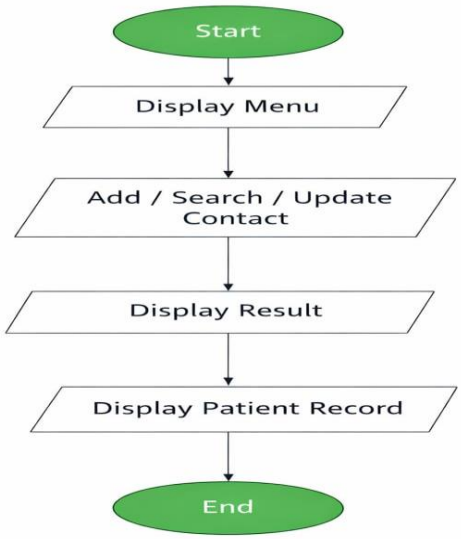
8.Repeat until user selects Exit

9.End the program

4.7. Flowchart

Students should draw a flowchart showing the program flow.

Example flow:



Flowcharts can be created using:

* draw.io
* Lucidchart
* MS Visio

5. SOURCE CODE

```
import java.util.Scanner;


class Contact {
    String name;
    String phone;




    String email;
    String address;
}


public class PersonalContactDirectory {


    static Contact contacts[] = new Contact[100];
    static int count = 0;


    public static void addContact(Scanner sc) {
        if (count>= 100) {
            System.out.println("Contact directory is full!");
            return;
        }
```

```
    Contact c = new Contact();

    System.out.print("Enter Name:");
    c.name = sc.nextLine();

    System.out.print("Enter Phone Number:");
    c.phone = sc.nextLine();

    System.out.print("Enter Email:");
    c.email = sc.nextLine();

    System.out.print("Enter Address:");
    c.address = sc.nextLine();

    contacts[count] = c;
    count++;

    System.out.println("Contact added successfully!");
}

public static void displayContacts() {
    if (count == 0) {
        System.out.println("No contacts available.");
```

Error! Number cannot be represented in specified format.

```
        return;
    }

    System.out.println("\n---- Contact List ----");
    for (int i = 0; i<count; i++) {
        System.out.println("Contact"+ (i + 1));
        System.out.println("Name:"+ contacts[i].name);
        System.out.println("Phone:"+ contacts[i].phone);
        System.out.println("Email:"+ contacts[i].email);
        System.out.println("Address:"+ contacts[i].address);
        System.out.println("----------------------");
    }
}

public static void searchContact(Scanner sc) {
    System.out.print("Enter name to search:");
    String searchName = sc.nextLine();

    boolean found = false;

    for (int i = 0; i<count; i++) {
        if (contacts[i].name.equalsIgnoreCase(searchName)) {
            System.out.println("Contact Found!");
```

```
            System.out.println("Name:"+ contacts[i].name);
            System.out.println("Phone:"+ contacts[i].phone);
            System.out.println("Email:"+ contacts[i].email);
            System.out.println("Address:"+ contacts[i].address);
            found = true;
            break;
        }
    }

    if (!found) {
        System.out.println("Contact not found.");
    }
}

public static void updateContact(Scanner sc) {
    System.out.print("Enter name of contact to update:");
    String name = sc.nextLine();

    for (int i = 0; i<count; i++) {
        if (contacts[i].name.equalsIgnoreCase(name)) {

            System.out.print("Enter new phonenumber:");
            contacts[i].phone = sc.nextLine();
```

```
            System.out.print("Enter new email:");
            contacts[i].email = sc.nextLine();

            System.out.print("Enter new address:");
            contacts[i].address = sc.nextLine();

            System.out.println("Contact updated successfully!");
            return;
        }
    }

    System.out.println("Contact not found.");
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    int choice;

    do {
        System.out.println("\n===== Personal Contact Directory =====");
        System.out.println("1. Add Contact");
        System.out.println("2. Display Contacts");
```

```
System.out.println("3. Search Contact");
System.out.println("4. Update Contact");
System.out.println("5. Exit");
System.out.print("Enter your choice:");
choice = sc.nextInt();
sc.nextLine();

switch (choice) {

    case 1:
        addContact(sc);
        break;

    case 2:
        displayContacts();
        break;

    case 3:
        searchContact(sc);
        break;

    case 4:
        updateContact(sc);
```

Error! Number cannot be represented in specified format.

```
                break;


            case 5:
                System.out.println("Exiting program...");
                break;
            default:
                System.out.println("Invalid choice!");
        }

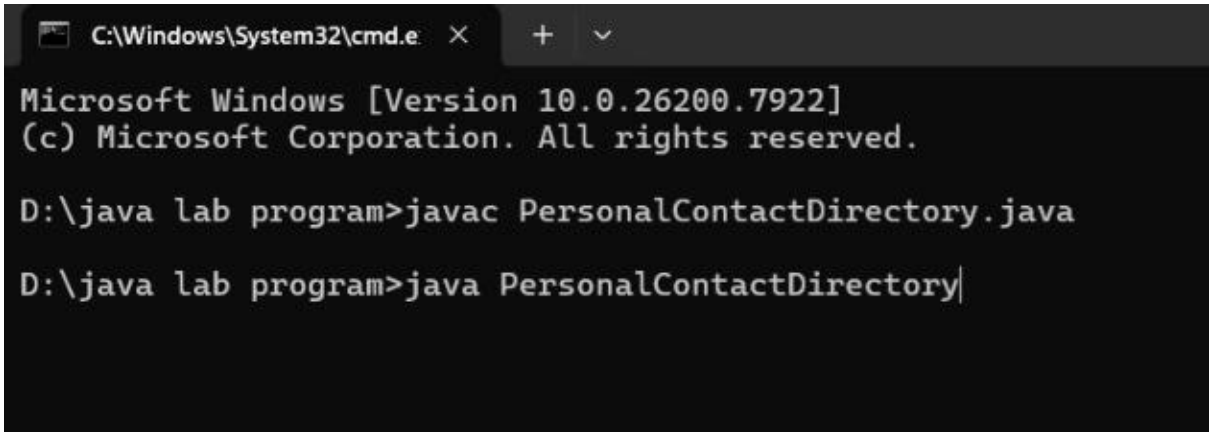

    } while (choice != 5);


    sc.close();
  }
}
```

Error! Number cannot be represented in specified format.

6.INPUT AND OUTPUT SCREENS

COMPILATION SCREEN:



INPUT SCREEN:



OUTPUT SCREEN:

Error! Number cannot be represented in specified format.

```
===== Personal Contact Directory =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Exit
Enter your choice: 2

---- Contact List ----
Contact 1
Name: Nihila
Phone: 9345780327
Email: nihila123@gmail.com
Address: 123,Happy Street,Coimbatore.
----------------------

===== Personal Contact Directory =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Exit
Enter your choice: 3
Enter name to search: Nihila
Contact Found!
Name: Nihila
Phone: 9345780327
Email: nihila123@gmail.com
Address: 123,Happy Street,Coimbatore.
```

```
===== Personal Contact Directory =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Exit
Enter your choice: 4
Enter name of contact to update: Nihila
Enter new phone number: 8668141176
Enter new email: nihila131@gmail.com
Enter new address: 31,Happy Street,RS Puram,Coimbatore.
Contact updated successfully!

===== Personal Contact Directory =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Exit
Enter your choice: 5
Exiting program...
```

Error! Number cannot be represented in specified format.

7.DISCUSSION&CONCLUSION

7.1 Discussion

The **Personal Contact Directory System** was developed to provide a simple solution for **managing personal contact details.**

The system allows users to store, search, update, and display contact information efficiently. Core Java programming concepts were used during development.

The system was tested with different inputs to ensure that it performs the required operations correctly.

The results show that the system simplifies the process of contact management.

7.2 Conclusion

The **Personal Contact Directory with Search and Update Features** successfully demonstrates the **use of Java programming** for developing a practical contact management application.

The system helps users store and manage contact information easily while reducing the effort required to s**earch and update contacts manually.**

The project also helps students understand how Java programming can be used to create simple real-world applications.

Future improvements can include:

• Database connectivity

• Graphical user interface

• Contact import/export features

8.REFERENCES

Books

• Herbert Schildt – Java: The Complete Reference

Websites

1. https://www.oracle.com/java

2. https://www.geeksforgeeks.org

3. https://www.w3schools.com

4. https://www.javatpoint.com